



UNIVERSIDADE DO VALE DO ITAJAÍ – UNIVALI
ESCOLA POLITÉCNICA
DISCIPLINA: SISTEMAS OPERACIONAIS

ANGELO MIGUEL REQUENHA
BRUNO DE QUEIROZ
LUCAS HENRIQUE DE MELO GUBERT

**SISTEMA DE PROCESSAMENTO PARALELO DE
REQUISIÇÕES A UM BANCO DE DADOS**

Avaliação M1 – IPC, Threads e Paralelismo

RESUMO

Este relatório descreve a implementação de um sistema que simula o funcionamento interno de um gerenciador de requisições a um banco de dados. O sistema é composto por dois executáveis independentes – *cliente* e *servidor* – que se comunicam exclusivamente por memória compartilhada e semáforos nomeados do sistema operacional. O servidor mantém um *pool* de threads POSIX (Pthreads) que consome as requisições de uma fila circular e acessa uma tabela de registros protegida por `pthread_mutex_t`. Foram realizadas simulações funcionais, de concorrência, de escalabilidade e de integridade, cujos resultados confirmam a correção das operações INSERT, SELECT, UPDATE e DELETE e a ausência de condições de corrida mesmo com dezenas de processos clientes simultâneos.

Palavras-chave: IPC; memória compartilhada; threads; exclusão mútua; produtor/consumidor.

1 INTRODUÇÃO E ENUNCIADO DO PROJETO

A avaliação M1 da disciplina de Sistemas Operacionais tem como objetivo consolidar os conceitos de comunicação entre processos (IPC), threads, concorrência e paralelismo. O enunciado propõe o desenvolvimento, em C/C++, de um **Sistema de Processamento Paralelo de Requisições a um Banco de Dados**, composto por:

- um **processo cliente**, que envia requisições de consulta ou alteração via IPC (pipe ou memória compartilhada);
- um **processo servidor**, que recebe as requisições e usa um **pool de threads** para processá-las em paralelo;
- uma estrutura compartilhada que simula o banco de dados (vetor ou arquivo), cujo acesso pelas threads é protegido por **mutex ou semáforo**.

As requisições suportadas devem ser INSERT (adiciona registro), DELETE (remove registro), SELECT (busca por ID) e UPDATE (modifica um campo por ID). Os requisitos técnicos obrigatórios exigem que cliente e servidor sejam executáveis distintos, rodando como processos separados; que a comunicação use IPC real; que o servidor use múltiplas threads; e que a tabela seja protegida por mutex ou semáforo real. O registro do banco segue a estrutura sugerida no enunciado:

Código 1 – Estrutura do registro (protocolo.h)

```
typedef struct {  
    int id;  
    char nome[TAM_NOME];    /* TAM_NOME = 50 */  
} Registro;
```

2 CONTEXTO E FUNDAMENTAÇÃO

Um sistema gerenciador de banco de dados precisa atender muitos clientes ao mesmo tempo sem corromper os dados. Isso envolve dois problemas clássicos de sistemas operacionais. O primeiro é a **comunicação entre processos**: como cliente e servidor possuem espaços de endereçamento isolados, eles não podem trocar dados por variáveis comuns; é necessário um mecanismo fornecido pelo núcleo, como pipes ou memória compartilhada (TANENBAUM; BOS, 2016). O segundo é a **exclusão mútua**: quando várias threads leem e escrevem a mesma estrutura, a intercalação arbitrária de instruções pode gerar condições de corrida – por exemplo, duas threads inserindo o mesmo ID ao mesmo tempo, ou uma remoção deslocando o vetor enquanto outra thread o percorre.

A solução adotada combina três técnicas clássicas: (i) memória compartilhada como canal de dados entre processos; (ii) o problema do **produtor/consumidor** com buffer limitado, resolvido com semáforos contadores, em que os clientes produzem requisições e as threads do servidor as consomem (SILBERSCHATZ; GALVIN; GAGNE, 2018); e (iii) uma **seção crítica** sobre a tabela, protegida por um `pthread_mutex_t`, garantindo que apenas uma thread manipule os registros por vez.

3 ARQUITETURA DA SOLUÇÃO

O código foi organizado em dois programas, cada um dividido em três arquivos por responsabilidade (interface, regra de negócio e comunicação), mais um cabeçalho comum, `protocolo.h`, que define o layout exato da memória compartilhada e os nomes dos objetos do sistema usados pelos dois lados.

Arquivo	Responsabilidade
<code>cabecalhos/protocolo.h</code>	Contrato entre os binários: structs, operações e layout da área compartilhada
<code>servidor/comunicacao.cpp</code>	Cria a memória compartilhada e os semáforos; fila circular; slots de resposta; pool de threads
<code>servidor/banco.cpp</code>	Tabela de registros, mutex, operações INSERT/SELECT/UPDATE/DELETE/LISTAR, persistência e log
<code>servidor/terminal.cpp</code>	Função main, argumentos (<code>--threads</code> , <code>--custo</code>) e saída em tela
<code>cliente/comunicacao.cpp</code>	Abre a memória e os semáforos pelo nome, envia a requisição e aguarda a resposta
<code>cliente/requisicao.cpp</code>	Converte o texto digitado (ex.: INSERT 7 Joao) em uma struct <code>Requisicao</code>
<code>cliente/terminal.cpp</code>	Função main, modo avulso e modo interativo

Tabela 1 – Organização dos arquivos-fonte

3.1 Mecanismos de IPC e sincronização

O sistema é executado nativamente no Windows. A camada entre processos usa os objetos nomeados do núcleo do Windows, equivalentes diretos aos da API POSIX; as threads e o mutex usam a própria API Pthreads, disponibilizada pela biblioteca *winpthreads* do compilador MinGW-w64.

Conceito	Implementação	Equivalente POSIX
Memória compartilhada	CreateFileMapping + MapViewOfFile	shm_open + mmap
Semáforos entre processos	CreateSemaphore (nomeado)	sem_open
Threads do pool	pthread_create / pthread_join	(mesma API)
Exclusão mútua na tabela	pthread_mutex_t	(mesma API)

Tabela 2 – Primitivas de sistema operacional utilizadas

3.2 Os dois canais de comunicação

Toda a comunicação ocorre na struct *AreaCompartilhada*, que contém dois canais. O **canal 1** é uma fila circular de 16 posições de requisições, controlada por três semáforos: vazios (posições livres), cheios (requisições pendentes) e fila (binário, exclusão mútua sobre cabeça e cauda). O **canal 2** é um vetor de 8 slots de resposta: cada requisição em andamento reserva um slot exclusivo e o cliente dorme no semáforo *pronta_<slot>* até o servidor depositar a resposta. Isso impede que dois clientes simultâneos recebam a resposta um do outro. Adicionalmente, cada atendimento é registrado com carimbo de tempo no arquivo *servidor.log*, e a tabela é persistida em *banco.txt*.

Código 2 – Layout da memória compartilhada (protocolo.h)

```
typedef struct {
    /* canal 1 - fila circular de requisicoes (produtor/consumidor) */
    Requisicao fila[TAM_FILA];
    int      cabeca;
    int      cauda;

    /* canal 2 - slots de resposta (poucos: so_m1 suporta alguns
     * clientes concorrentes, o suficiente para demonstrar o pool) */
    Resposta resposta[MAX_SLOTS];
    int      pilha_livres[MAX_SLOTS];
    int      topo_livres;

    int servidor_ativo;
    int num_threads;
} AreaCompartilhada;
```

O fluxo completo de uma requisição é: (1) o cliente reserva um slot livre; (2) insere a requisição na fila circular e sinaliza cheios; (3) uma thread livre do pool retira a requisição; (4) a thread trava o mutex da tabela, executa a operação e destrava; (5) a thread grava a resposta no slot e sinaliza *pronta_<slot>*; (6) o cliente acorda, lê a resposta e devolve o slot.

4 CÓDIGOS IMPORTANTES DA IMPLEMENTAÇÃO

O Código 3 mostra o lado produtor, executado pelo processo cliente. A espera em `sem_vazios` bloqueia o cliente quando a fila está cheia, e o semáforo binário `sem_fila` protege o índice `cauda`, que é disputado por vários clientes.

Código 3 – Envio de requisição pelo cliente (cliente/comunicacao.cpp)

```
static void enviar(Requisicao* r)
{
    r->pid_cliente = (int)GetCurrentProcessId();

    esperar(sem_vazios);
    esperar(sem_fila);

    area->fila[area->cauda] = *r;
    area->cauda = (area->cauda + 1) % TAM_FILA;

    liberar(sem_fila);
    liberar(sem_cheios);
}
```

No servidor, cada thread do pool executa o laço do Código 4: retira uma requisição (lado consumidor), processa e responde no slot indicado. O processamento ocorre fora da seção crítica da fila, de modo que outras threads continuam retirando requisições enquanto uma delas acessa a tabela.

Código 4 – Consumo da fila e laço das threads do pool (servidor/comunicacao.cpp)

```

static bool retirar(Requisicao* saida)
{
    esperar(sem_cheios);
    if (!area->servidor_ativo) return false;

    esperar(sem_fila);
    *saida = area->fila[area->cabeca];
    area->cabeca = (area->cabeca + 1) % TAM_FILA;
    liberar(sem_fila);

    liberar(sem_vazios);
    return true;
}

static void* trabalhador(void* arg)
{
    int id = (int)(intptr_t)arg;
    log_escrever("POOL thread %d pronta", id);

    Requisicao req;
    while (retirar(&req)) {
        log_escrever("RECV thread %d <- pid %d op %d id %d",
                    id, req.pid_cliente, (int)req.op, req.id);

        Resposta res = banco_processar(req, id);
        responder(req.slot, &res);

        log_escrever("SEND thread %d -> %s | %s",
                    id, res.ok ? "OK " : "ERR", res.texto);

        if (req.op == OP_ENCERRAR) {
            ipc_pedir_parada();
            break;
        }
    }

    log_escrever("POOL thread %d encerrada", id);
    return nullptr;
}

```

O Código 5 mostra a criação do pool e a proteção da tabela. Toda operação sobre os registros é envolvida por `pthread_mutex_lock/pthread_mutex_unlock` sobre o mesmo mutex. O parâmetro `--custo` introduz um atraso artificial dentro da seção crítica, simulando o tempo de E/S de um banco real e tornando a exclusão mútua observável no log.

Código 5 – Criação do pool e seção crítica sobre a tabela (servidor)

```
void ipc_rodar_pool(void)
{
    pthread_t th[TAM_FILA];

    for (int i = 0; i < total_threads; i++)
        pthread_create(&th[i], nullptr, trabalhador, (void*)(intptr_t)i);

    for (int i = 0; i < total_threads; i++)
        pthread_join(th[i], nullptr);
}

static pthread_mutex_t mtx_tabela = PTHREAD_MUTEX_INITIALIZER;

case OP_INSERT:
    pthread_mutex_lock(&mtx_tabela);
    gastar_tempo();
    fazer_insert(req, res);
    pthread_mutex_unlock(&mtx_tabela);
    break;
...

```

Por fim, o servidor cria os objetos nomeados e detecta se já existe outra instância em execução (Código 6). Os valores iniciais dos semáforos definem a capacidade da fila e o número de slots.

Código 6 – Inicialização dos semáforos (servidor/comunicacao.cpp)

```
sem_vazios = criar_semaforo(NOME_VAZIOS, TAM_FILA, TAM_FILA);
sem_cheios = criar_semaforo(NOME_CHEIOS, 0, TAM_FILA);
sem_fila = criar_semaforo(NOME_FILA, 1, 1);
sem_slots = criar_semaforo(NOME_SLOTS, MAX_SLOTS, MAX_SLOTS);
sem_pilha = criar_semaforo(NOME_PILHA, 1, 1);

for (int i = 0; i < MAX_SLOTS; i++) {
    char nome[64];
    snprintf(nome, sizeof nome, "%s%d", NOME_PRONTA, i);
    sem_pronta[i] = criar_semaforo(nome, 0, 1);
    area->pilha_livres[i] = i;
}

```

5 RESULTADOS OBTIDOS COM AS SIMULAÇÕES

Os testes foram executados em Windows 11 Pro, com os binários compilados pelo g++ do MinGW-w64 (variante POSIX) sem avisos de compilação (-Wall -Wextra). Um script disparou os processos clientes reais contra o servidor, registrando as respostas, os tempos e o servidor.log. O banco foi inicializado com cinco registros (IDs 1 a 5) antes de cada cenário.

5.1 Teste funcional

A Tabela 3 apresenta uma sequência de requisições enviadas pelo cliente, cada uma por um processo distinto, com um pool de 4 threads. Todas as operações retornaram o resultado esperado, incluindo os casos de erro (ID duplicado e ID inexistente). Nota-se também que as requisições foram distribuídas entre as quatro threads do pool.

Requisição	Resposta do servidor
LISTAR	[OK] (thread 0) LISTAR → 5 registro(s) na tabela
SELECT 5	[OK] (thread 2) SELECT id=5 → nome=Pedro
INSERT 7 Joao Silva	[OK] (thread 3) INSERT id=7 nome=Joao Silva → ok
INSERT 7 Outro	[ERR] (thread 1) INSERT id=7 → id ja existe
SELECT 7	[OK] (thread 0) SELECT id=7 → nome=Joao Silva
UPDATE 7 Maria	[OK] (thread 2) UPDATE id=7 → nome=Maria
SELECT 7	[OK] (thread 3) SELECT id=7 → nome=Maria
DELETE 7	[OK] (thread 1) DELETE id=7 → removido
SELECT 7	[ERR] (thread 0) SELECT id=7 → nao encontrado
UPDATE 99 X	[ERR] (thread 2) UPDATE id=99 → nao encontrado
DELETE 99	[ERR] (thread 3) DELETE id=99 → nao encontrado
LISTAR	[OK] (thread 1) LISTAR → 5 registro(s) na tabela

Tabela 3 – Sequência de requisições e respostas (pool = 4 threads)

5.2 Concorrência entre clientes e exclusão mútua

Neste cenário, três processos clientes foram iniciados ao mesmo tempo (SELECT 1, INSERT 8 Bruno e UPDATE 2 Mariana), com 4 threads no servidor e custo artificial de 500 ms por operação. O trecho do log abaixo mostra que as três requisições foram **recebidas quase simultaneamente** por três threads diferentes, mas **concluídas em intervalos de aproximadamente 500 ms**: cada thread precisou esperar a liberação do mutex para acessar a tabela.

Código 7 – Trecho do servidor.log (3 clientes simultâneos, custo = 500 ms)

```
[23:06:33.533] RECV thread 1 <- pid 17804 op 2 id 1
[23:06:33.545] RECV thread 0 <- pid 16288 op 1 id 8
[23:06:33.567] RECV thread 2 <- pid 3036 op 3 id 2
[23:06:34.076] SEND thread 1 -> OK | SELECT id=1 -> nome=Joao
[23:06:34.592] SEND thread 0 -> OK | INSERT id=8 nome=Bruno -> ok
[23:06:35.100] SEND thread 2 -> OK | UPDATE id=2 -> nome=Mariana
```

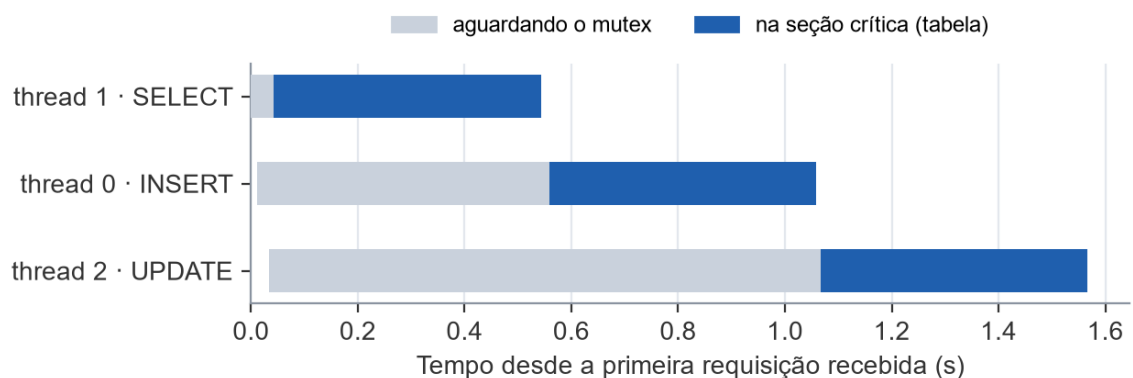


Figura 1 – Linha do tempo das threads, reconstruída a partir do servidor.log

5.3 Variação do tamanho do pool

Para avaliar o comportamento com diferentes tamanhos de pool, oito processos clientes enviaram requisições SELECT simultaneamente, com custo de 200 ms por operação. Cada configuração foi executada três vezes. A Tabela 4 e a Figura 2 apresentam os tempos totais medidos, do disparo do primeiro cliente até a resposta do último.

Threads no pool	Execução 1 (s)	Execução 2 (s)	Execução 3 (s)	Média (s)	Threads que atenderam
1	1,655	1,653	1,665	1,658	0
2	1,651	1,656	1,684	1,664	0, 1
4	1,671	1,669	1,689	1,676	0, 1, 2, 3
8	1,668	1,670	1,645	1,661	0, 1, 2, 3, 4, 5, 6, 7

Tabela 4 – Tempo total para atender 8 clientes simultâneos (custo = 200 ms)

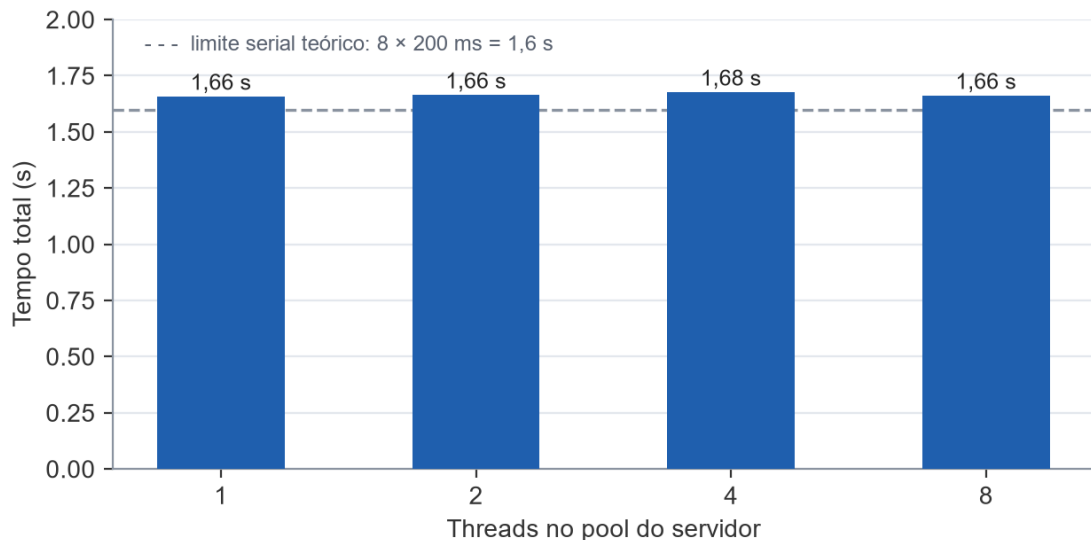


Figura 2 – Tempo médio total em função do número de threads

5.4 Integridade dos dados sob concorrência

O último cenário verifica a ausência de condições de corrida. Com 8 threads e custo zero, foram disparados 200 processos clientes executando INSERT com IDs distintos, em lotes de 25 processos simultâneos. Em seguida, 16 processos tentaram inserir **o mesmo ID** (5000) ao mesmo tempo.

Verificação	Esperado	Obtido
INSERTs com ID distinto aceitos	200	200
Registros na tabela após o teste (LISTAR)	205	205
INSERTs simultâneos do mesmo ID aceitos	1	1
INSERTs simultâneos do mesmo ID rejeitados	15	15
Linhas gravadas em banco.txt	206	206

Verificação	Esperado	Obtido
IDs únicos em banco.txt	206	206
Tempo para os 200 INSERTs	–	2,14 s

Tabela 5 – Resultados do teste de integridade (pool = 8 threads)

6 ANÁLISE E DISCUSSÃO DOS RESULTADOS

O teste funcional (Tabela 3) demonstra que as quatro operações exigidas – INSERT, SELECT, UPDATE e DELETE – funcionam corretamente, incluindo as situações de erro. Cada requisição foi enviada por um processo de sistema operacional diferente, como mostram os PIDs distintos registrados no log, o que confirma que cliente e servidor são executáveis separados comunicando-se apenas por IPC.

O cenário de concorrência (Código 7 e Figura 1) evidencia dois comportamentos complementares. Primeiro, o **pool de threads trabalha em paralelo**: as três requisições foram retiradas da fila por três threads distintas em um intervalo de apenas 34 ms. Segundo, a **exclusão mútua funciona**: embora as threads estejam ativas ao mesmo tempo, apenas uma por vez permanece na seção crítica, e as conclusões ocorrem espaçadas em aproximadamente 500 ms, exatamente o custo configurado.

A variação do tamanho do pool (Tabela 4 e Figura 2) mostra que o tempo total permaneceu próximo de 1,66 s em todas as configurações, muito perto do limite teórico de $8 \times 200 \text{ ms} = 1,6 \text{ s}$. Isso ocorre porque o custo simulado está dentro da seção crítica: como todas as operações, inclusive as leituras, utilizam o mesmo mutex, o trabalho sobre a tabela é inerentemente serial, e aumentar o número de threads não reduz o tempo – um exemplo prático da Lei de Amdahl. A coluna "Threads que atenderam" confirma, porém, que todas as threads do pool participaram do atendimento. Ganho de desempenho real exigiria reduzir a parte serial, por exemplo com o padrão leitores/escritores (várias leituras SELECT simultâneas via `sem_t` ou `pthread_rwlock_t`), possibilidade citada no enunciado como alternativa, ou com travas de granularidade mais fina.

O teste de integridade (Tabela 5) é o resultado mais importante do ponto de vista de sistemas operacionais. Com 200 inserções concorrentes, nenhum registro foi perdido ou duplicado, e o arquivo `banco.txt` terminou com exatamente 206 IDs únicos. No teste de inserção simultânea do mesmo ID, apenas uma requisição foi aceita e as outras quinze foram rejeitadas: a verificação "ID já existe" e a inserção ocorrem de forma atômica dentro do mutex, eliminando a condição de corrida do tipo *check-then-act*. Sem o mutex, duas threads poderiam passar pela verificação antes de qualquer uma inserir, gerando registros duplicados.

Entre as limitações conhecidas, destacam-se a gravação completa do arquivo `banco.txt` dentro da seção crítica a cada alteração, o que aumenta o tempo com o mutex travado, e a busca linear por ID, adequada ao limite de 1024 registros adotado.

7 CONCLUSÃO

O sistema desenvolvido atende à proposta da avaliação: dois processos independentes trocam requisições e respostas por memória compartilhada sincronizada com semáforos, o servidor distribui o trabalho entre um pool de threads Pthreads e a tabela compartilhada é protegida por `pthread_mutex_t`. As simulações confirmaram a correção das operações, o funcionamento paralelo do pool e a integridade dos dados sob alta concorrência. A análise de desempenho mostrou, na prática, como a seção crítica determina o limite de paralelismo de um sistema concorrente, apontando o uso de leitores/escritores como evolução natural do trabalho.

REFERÊNCIAS

MICROSOFT. **Creating named shared memory**. Microsoft Learn, 2023. Disponível em: <https://learn.microsoft.com/windows/win32/memory/creating-named-shared-memory>. Acesso em: 22 set. 2026.

MICROSOFT. **Semaphore objects**. Microsoft Learn, 2023. Disponível em: <https://learn.microsoft.com/windows/win32/sync/semaphore-objects>. Acesso em: 22 set. 2026.

SILBERSCHATZ, A.; GALVIN, P. B.; GAGNE, G. **Operating system concepts**. 10. ed. Hoboken: Wiley, 2018.

TANENBAUM, A. S.; BOS, H. **Sistemas operacionais modernos**. 4. ed. São Paulo: Pearson, 2016.

THE OPEN GROUP. **pthread_mutex_lock**. The Open Group Base Specifications Issue 7, 2018. Disponível em: <https://pubs.opengroup.org/onlinepubs/9699919799/>. Acesso em: 22 set. 2026.